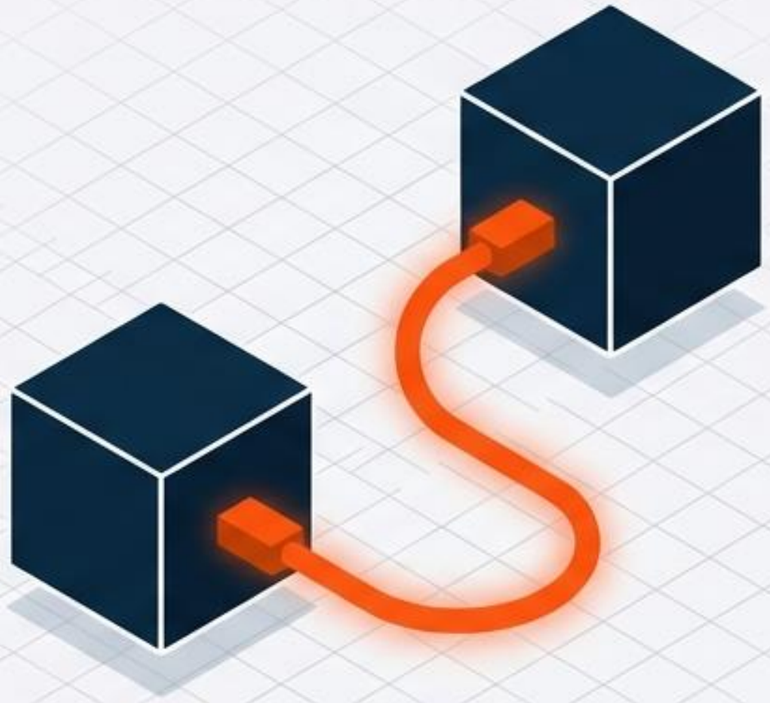
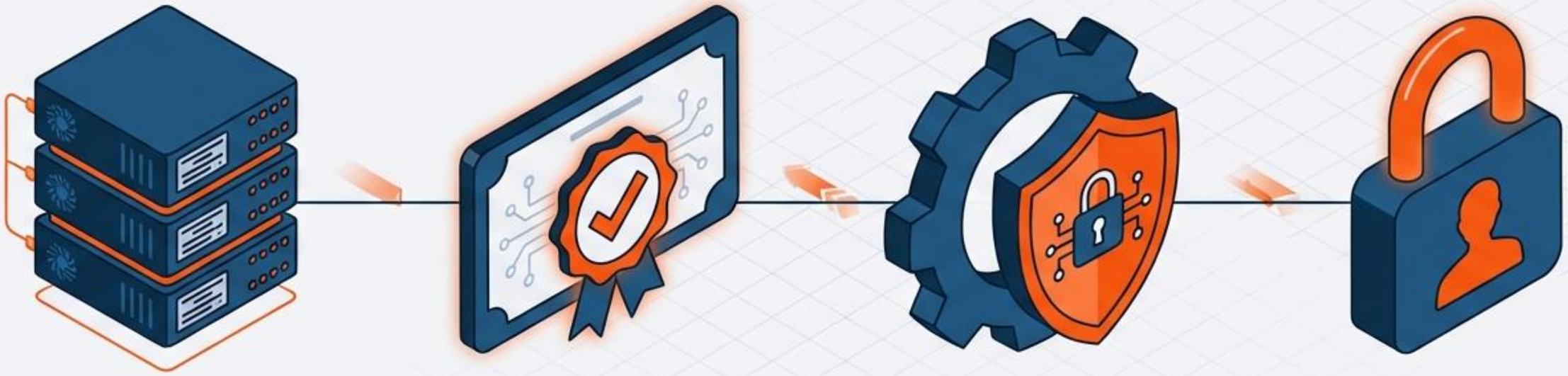


Sécurisation d’une Connexion MQTT

Guide de Mise en Œuvre : Infrastructure, Chiffrement TLS et Authentification

FORMATION IOT SECURITY / TRAVAUX PRATIQUES

Objectifs du TP



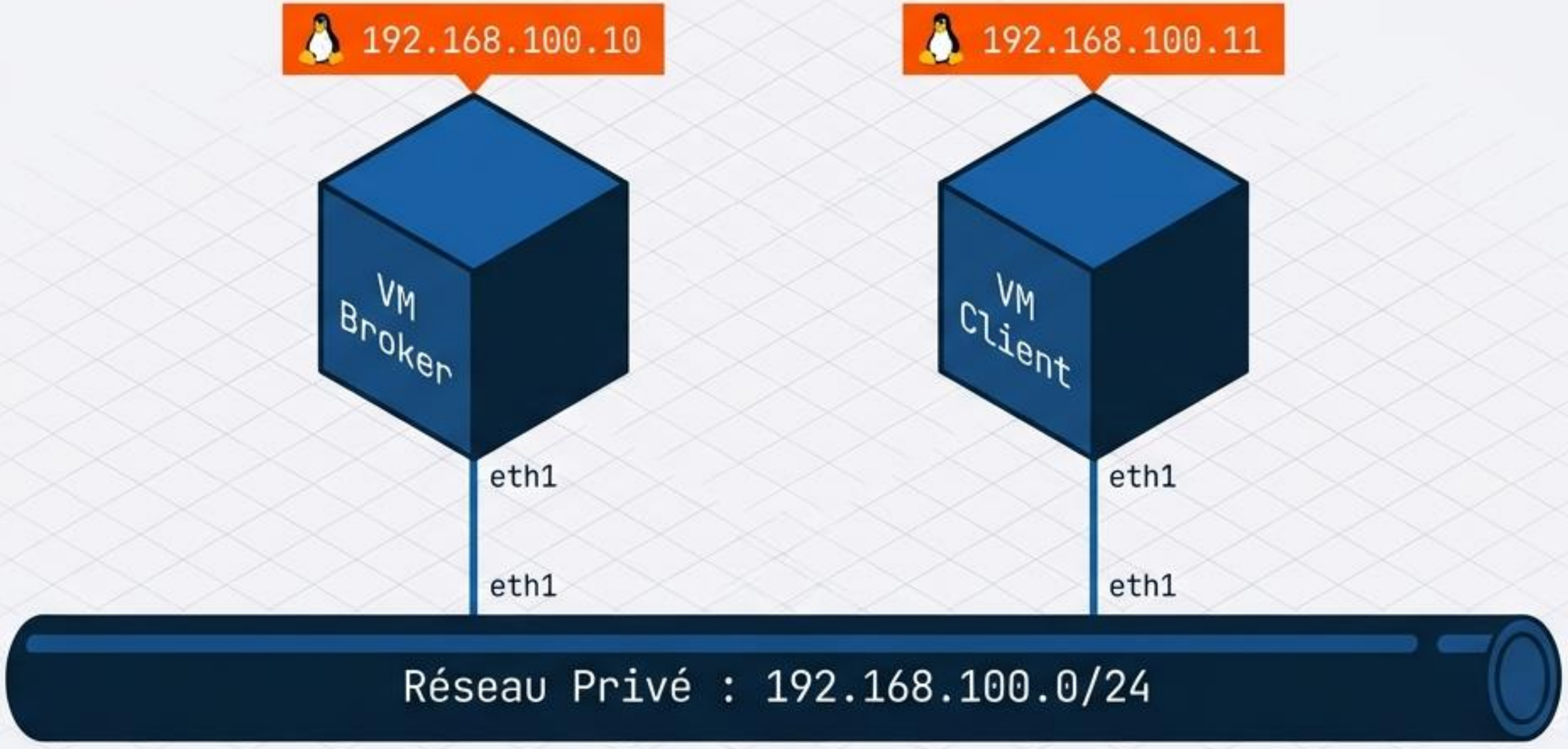
1. Infrastructure
(KVM & Vagrant)

2. Identité
(PKI & SSL/TLS)

3. Sécurisation
(Broker MQTTS)

4. Contrôle
(Authentification)

Topologie de l’Infrastructure



Orchestration : **Vagrant + Libvirt (KVM)**

Déploiement Automatisé

1. Clonage du dépôt :

```
git clone https://github.com/bouhenic/FormationIOTSecu
```

2. Lancement :

```
vagrant up --provider=libvirt
```

3. Vérification :

```
vagrant status
```

Vagrantfile Config

- **OS Base :** Debian Bookworm
- **Ressources :** 2 vCPUs, 2048MB RAM
- **Provisioning :** Shell (install mosquitto, openssl)

```
Current machine states:

  mosquitto-broker     running (libvirt)
  mosquitto-client     running (libvirt)

This environment represents multiple VMs. The VMs are
all listed above with their current state. For more
information about a specific VM, run vagrant status
NAME.
```

La Hiérarchie de Confiance (Certificate Authority)

Production (Confiance Mondiale)

- Exemples : Verisign, Let's Encrypt
- Reconnu par tous les navigateurs et OS.

Lab / Développement (Confiance Locale)

- Exemple : Self-Signed (Auto-signé)
- Nous sommes notre propre autorité.
- Nécessite d'installer manuellement le certificat racine.

Génération du CA (Racine de Confiance)

Génère un certificat auto-signé

```
openssl req -new -x509 -days 1826 -extensions
v3_ca -keyout ca.key -out ca.crt
```

Clé Privée (SECRET)

Certificat Public (PUBLIC)

Validité de 5 ans

Identité du Serveur : La Clé Privée

server.key

La première étape pour identifier le Broker est de générer sa clé privée RSA.

```
openssl genrsa -out server.key 2048
```

Technical Breakdown

- Algorithme : RSA
- Taille : 2048 bits
- Usage : Signature et déchiffrement

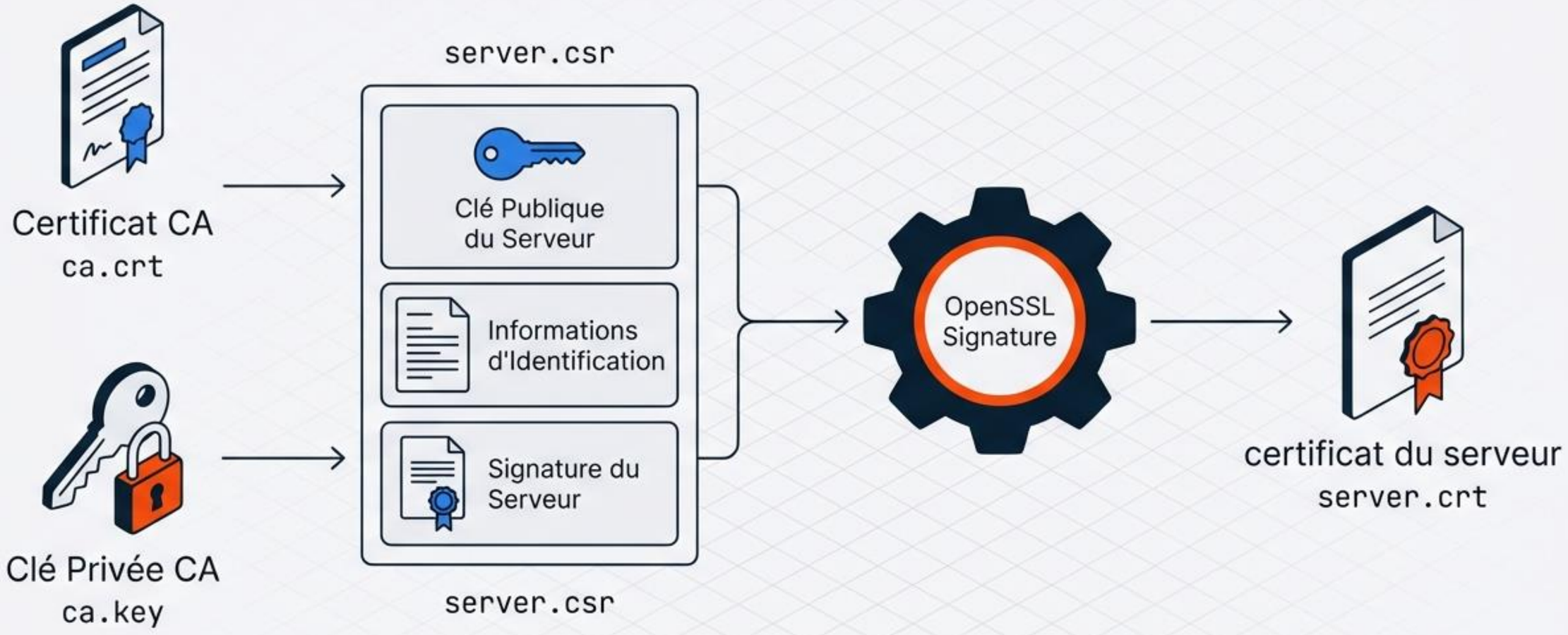
La Demande de Signature (CSR)

```
openssl req -out server.csr -key server.key -new
```

```
Country Name (2 letter code) [AU]: FR
State or Province Name (full name) [Some-State]: [Enter]
Locality Name (eg, city) []: PARIS
Organization Name (eg, company) [Internet Widgits Pty Ltd]: Newton
Organizational Unit Name (eg, section) []: CIEL
Common Name (e.g. server FQDN or YOUR name) []: 192.168.100.10
```

Important : Doit correspondre à l'IP du Broker

Le Processus de Signature



Génération du Certificat Signé

```
openssl x509 -req \
   -in server.csr \
   -CA ca.crt \
   -CAkey ca.key \
   -CAcreateserial \
   -out server.crt \
   -days 360
```

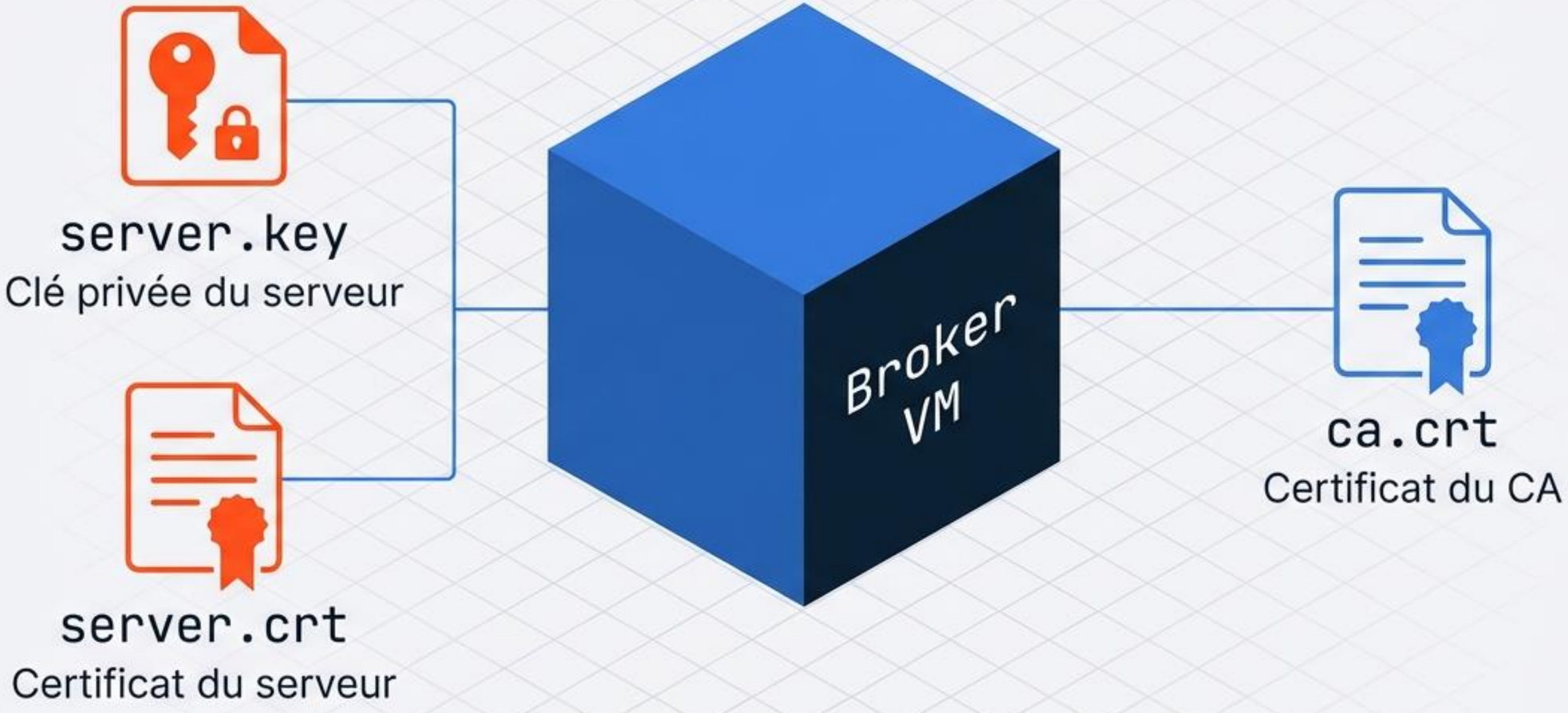
`-CAcreateserial` Crée un numéro de série unique.

`-CAkey` Signe avec la clé privée de l'autorité.

`-out server.crt` Génère le certificat final pour le broker.

Cette commande transforme la demande (CSR) en identité validée.

Inventaire des Fichiers de Sécurité



Configuration du Broker Mosquitto

Fichier : /etc/mosquitto/mosquitto.conf

```
listener 8883


cafile   /home/vagrant/ca.crt
certfile /home/vagrant/server.crt
keyfile  /home/vagrant/server.key
```

Port MQTTS (Sécurisé)

Chemins absolus vers les clés générées

Activation du Service

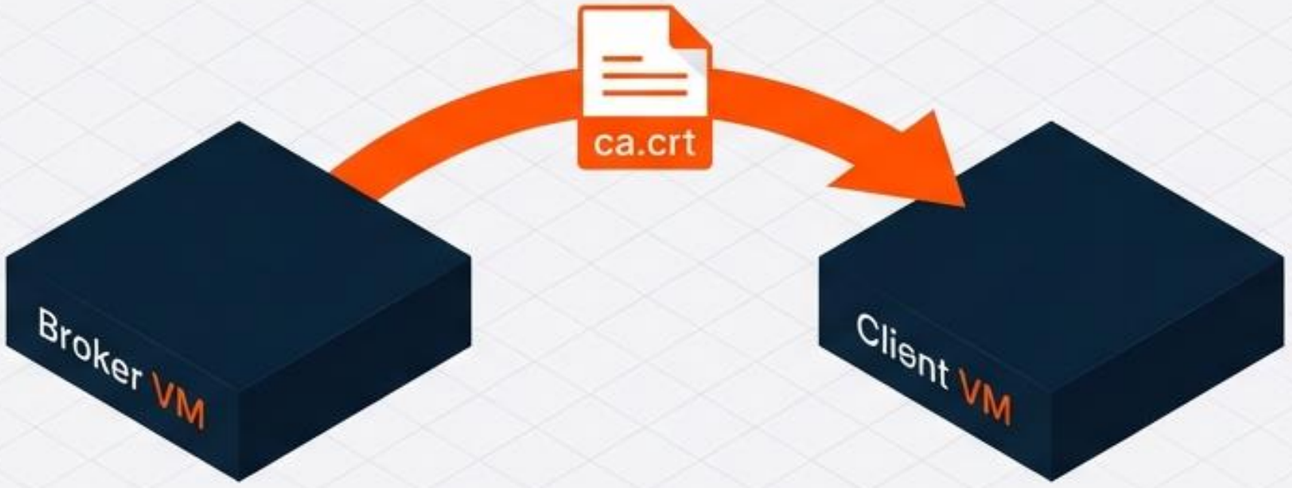
```
>_ sudo systemctl restart mosquitto
```

```
>_ sudo systemctl status mosquitto
```

```
● mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/lib/systemd/system/mosquitto.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2024-10-27 14:30:00 UTC; 2min ago
 Main PID: 1234 (mosquitto)
    Tasks: 3 (limit: 4915)
   CGroup: /system.slice/mosquitto.service
           └─1234 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf
```

Si le service ne redémarre pas, vérifiez les permissions des fichiers certificats.

Établissement de la Confiance Client



Le client doit posséder le certificat public du CA pour valider l'identité du serveur.

```
›_ cat /home/vagrant/ca.crt
```

Sur le Broker - Copier

```
›_ nano /home/vagrant/ca.crt
```

Sur le Client - Coller

Test de Connexion MQTTS

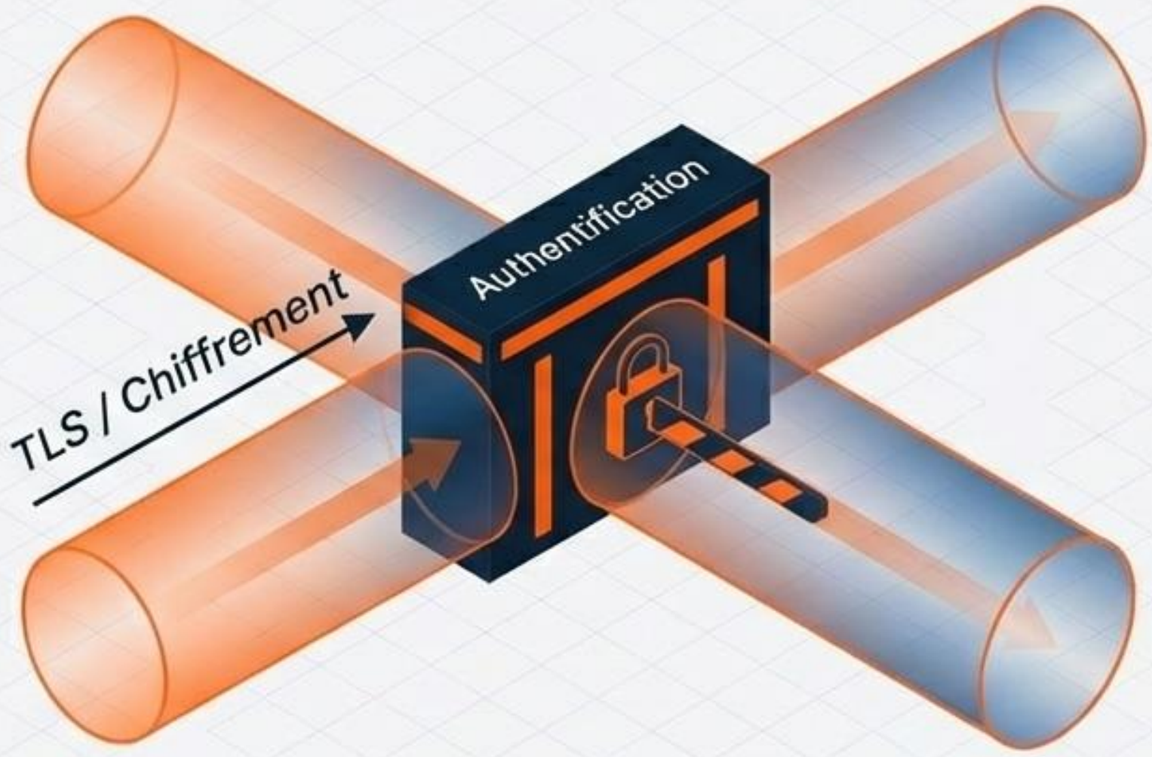
```
>_

mosquitto_sub -h 192.168.100.10 -p 8883 --cafile ca.crt -t
test/topic
```

```
>_

mosquitto_pub -h 192.168.100.10 -p 8883 --cafile ca.crt -t
 test/topic -m "Hello Secure World"
```

Niveau 2 : Authentification Utilisateur



⚠️ Actuellement : `allow_anonymous true` (Tout le monde peut entrer).

✅ Objectif : `allow_anonymous false` (Badge d'identité requis).

Création des Utilisateurs

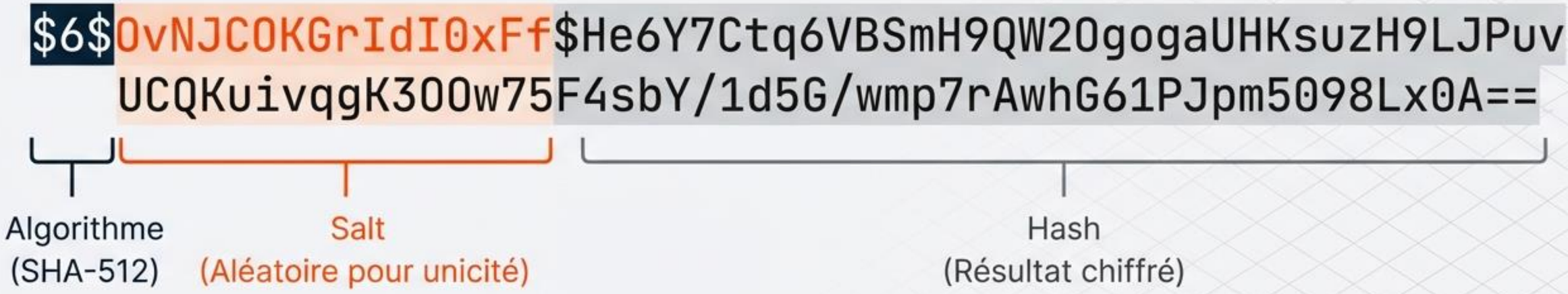
```
>_ sudo mosquitto_passwd -c /mosquitto_passwd userclient
```

```
>_ sudo mosquitto_passwd /mosquitto_passwd userbroker
```

```
Password: ********
Reenter password: ********
```

Note : L'option `-c` crée un nouveau fichier (écrase l'existant). Pour le second utilisateur, on omet `-c`.

Analyse du Hachage (Security Anatomy)



Les mots de passe ne sont jamais stockés en clair.

Durcissement de la Configuration

Fichier : `/etc/mosquitto/mosquitto.conf`

```
~~allow_anonymous true~~

allow_anonymous false
password_file /mosquitto_passwd
```

Pointe vers la base de données hachée.

- Redémarrer le service.

Validation Finale (TLS + Auth)

```
mosquitto_sub -h 192.168.100.10 -p 8883
--cafile ca.crt -u userclient -P [password] -t topic
```

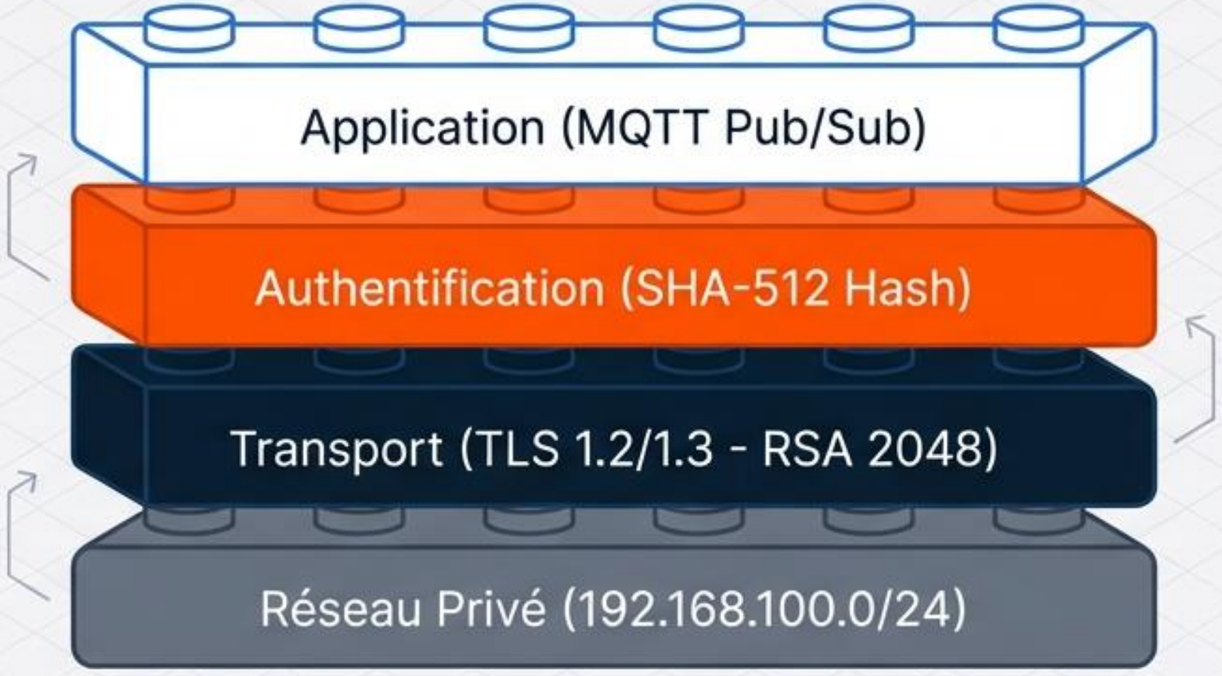
Canal Sécurisé (TLS)

Identité Serveur (Cert)

Identité Client (Auth)

CONNEXION RÉUSSIE

Architecture Sécurisée : Vue d'Ensemble



Une forteresse construite brique par brique.